

MODEL-AIDED DEEP LEARNING FOR SIGNAL PROCESSING WITH FERROELECTRET-BASED FORCE SENSORS

Hao Yuan

Communication Systems Group

CONTENTS

1

Introduction

2

Data collection

3

Proposed Estimation Methods

4

Experimental Result

5

Conclusion

INTRODUCTION

Section 1

BACKGROUND

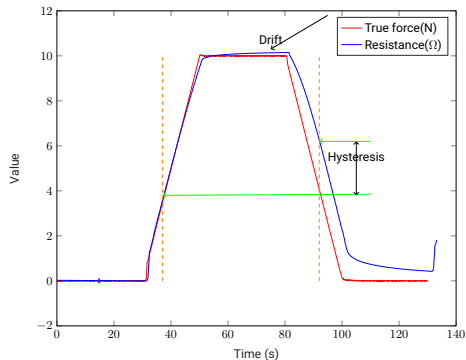
FORCE SENSORS

- Applications
 - Robotics
 - Human-machine interfaces
- Conventional sensors
 - Load cells
 - Electrical Resistance Strain Gauges (ERSGs)
- Limitations
 - Costly
 - Bulky and rigid
 - Drift and hysteresis error



<https://mt.rsdelivers.com/>

Hysteresis and drift

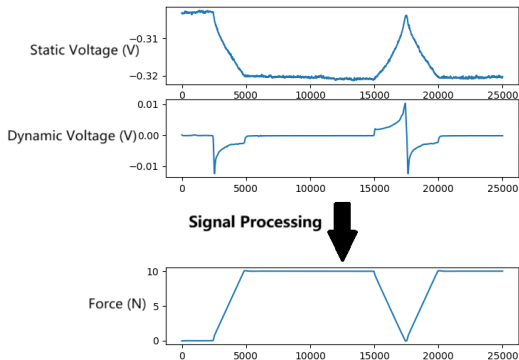


BACKGROUND

FERROELECTRET-BASED FORCE SENSORS



- Specificities
 - Light-weight
 - Cost effective
 - Two measurements
 - Static Voltage
 - Dynamic Voltage
- Principle
 - Electrical resistance strain gauge
 - Piezoelectric



POTENTIAL APPROACHES

Deep learning

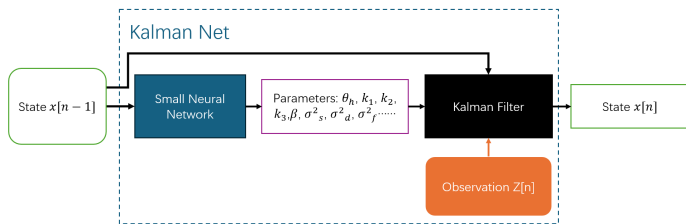
- Applications
 - Computer vision
 - NLP
- Specificities
 - Data-driven
 - Black box
- Pros and Cons
 - ✓ Complex tasks
 - ✓ Generalization
 - ✗ Bad Interpretability
 - ✗ Large quantities of parameters

Kalman filter

- Applications
 - Sensor fusion
 - Guidance and navigation
- Specificities
 - Model-based
 - Optimal Estimation
- Pros and Cons
 - ✓ Partially interpretable
 - ✓ Real-time implementation
 - ✓ Good Robustness
 - ✗ Insufficient non-linearity
 - ✗ High dependence on physical models

DEEP LEARNED KALMAN FILTER

- Specificities
 - Hybrid
 - Data-driven
 - Model-aided
- Advantages
 - ✓ Good generalization
 - ✓ Interpretable
 - ✓ Computationally efficient
 - ✓ Light weight



CONTENTS

1

Introduction

2

Data collection

3

Proposed Estimation Methods

4

Experimental Result

5

Conclusion

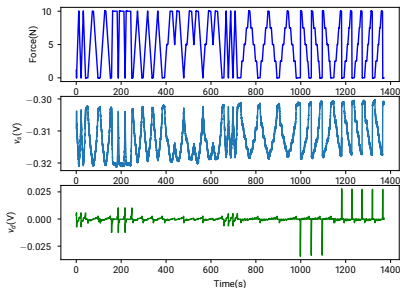
DATA COLLECTION

Section 2

DATA COLLECTION

- Universal testing machine
 - Programmable
 - Reference force value
- Mimosense Electronics
 - Sensor: SDK002
 - Static voltage V_s
 - Dynamic voltage V_d
- Measurement result
 - Observation vector $\mathbf{z}$

$$\mathbf{z} = \begin{bmatrix} V_s \\ V_d \end{bmatrix} \quad (1)$$

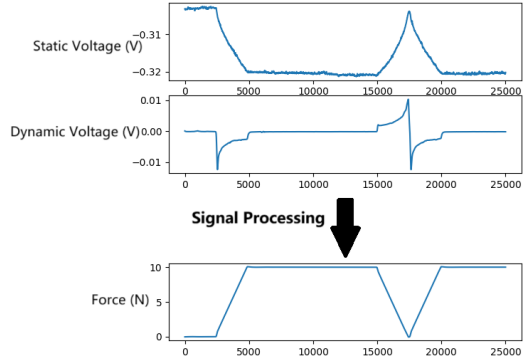


DATA PREPROCESSING

- Low pass filter
 - 8th-order Butterworth low-pass filter
 - Cutoff frequency f_c

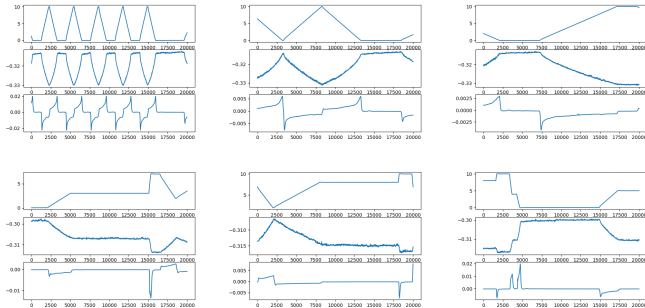
$$f_c = 0.08 \frac{f_s}{2}$$

- Synchronization
 - Why out of sync?
 - Solution: First-order derivative matching



DATA SLICING

- Segments of length $S = 20000$ samples
- Data set
 - Training set
 $|\mathcal{D}_{\text{train}}| = 89$
 - Validation set
 $|\mathcal{D}_{\text{validation}}| = 3$
 - Unsliced validation sequence



CONTENTS

1
2
3

Introduction

Data collection

Proposed Estimation Methods

4
5

Experimental Result

Conclusion

PROPOSED ESTIMATION METHODS

Section 3

OPTIMAL MODEL FOR ESTIMATION

- Measurement vector $\mathbf{z}$
- Force estimation based on Model

$$\hat{f}[n_0] = \mathcal{M}((\mathbf{z}[n])_{n=1}^{n_0}, \mathbf{w}) \quad (2)$$

- Optimal the weights are obtained by Stochastic Gradient Descent(SGD) with loss function $\mathcal{L}$

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathcal{L}(f, \hat{f}) \quad (3)$$

- The goal is to find model $\mathcal{M}$ with optimal weights $\mathbf{w}^*$

$$f[n_0] \approx \mathcal{M}((\mathbf{z}[n])_{n=1}^{n_0}, \mathbf{w}^*) \quad (4)$$

KALMAN FILTER

- State model

$$\mathbf{x}[n] = [x_1[n], x_2[n], x_3[n], \dots]^T$$

$$\mathbf{z}[n] = [z_1[n], z_2[n], \dots]^T \text{ with uncertainty } \mathbf{v}[n]$$

$$\mathbf{R}[n] = E[\mathbf{v}[n]^T \mathbf{v}[n]] = I[\sigma_1^2, \sigma_2^2, \dots]^T$$

- State transition
 - Prediction

- Predicted state:

$$\mathbf{x}[n] = \mathbf{A}[n]\mathbf{x}[n-1] + \mathbf{w}[n] + \mathbf{B}[n]\mathbf{u}[n] \quad (5)$$

- Covariance of the process noise:

$$\mathbf{Q}[n] = E[\mathbf{w}[n]^T \mathbf{w}[n]] \quad (6)$$

- Covariance of the predicted state:

$$\mathbf{P}[n] = \mathbf{A}[n]\mathbf{P}[n-1]\mathbf{A}[n]^T + \mathbf{Q}[n] \quad (7)$$

KALMAN FILTER

- State transition
 - Update
 - Predicted measurement:

$$\mu[n] = \mathbf{H}[n]\mathbf{x}[n] \quad (8)$$

- Covariance of the predicted measurement:

$$\Sigma = \mathbf{H}[n]\mathbf{P}[n]\mathbf{H}[n]^T \quad (9)$$

- Kalman gain:

$$\mathbf{K}[n] = \mathbf{P}[n]\mathbf{H}[n]^T (\mathbf{H}[n]\mathbf{P}[n]\mathbf{H}[n]^T + \mathbf{R}[n])^{-1} \quad (10)$$

- State update:

$$\mathbf{x}[n]' = \mathbf{x}[n] + \mathbf{K}[n] (\mathbf{z}[n] - \mu[n]) \quad (11)$$

- Covariance update:

$$\mathbf{P}[n]' = (\mathbf{I} - \mathbf{K}[n]\mathbf{H}[n]) \mathbf{P}[n] \quad (12)$$

PROPOSED MODEL

- State model

$$\mathbf{x}[n] = [f[n], \Delta\ell[n], \Delta\ell[n-1], \Delta\ell[n-2], h[n]]^T \quad (13)$$

$$\begin{aligned} f[n] &= k_1[n]\Delta\ell[n] + k_2[n](\Delta\ell[n] - \Delta\ell[n-1]) \\ &\quad + k_3[n](\Delta\ell[n] - 2\Delta\ell[n-1] + \Delta\ell[n-2]) \end{aligned} \quad (14)$$

$$\mathbf{z}[n] = [v_s[n], v_d[n]]^T \quad (15)$$

$$\mathbf{R}[n] = I[\sigma_s^2, \sigma_d^2]^T \quad (16)$$

- State transition

- Force

$$f[n] = f[n-1] + \Delta f[n] \quad (17)$$

- Change in length

$$\Delta\ell[n] = \left(\frac{1}{k_1[n] + k_2[n] + k_3[n]} \right) [f[n-1] + (k_2 + 2k_3)\Delta\ell[n-1] + k_3\Delta\ell[n-2]] \quad (18)$$

PROPOSED MODEL(CONT.)

- State transition
 - Change in length

$$\Delta\ell[n-1] = \Delta\ell[n], \quad (19)$$

$$\Delta\ell[n-2] = \Delta\ell[n-1] \quad (20)$$

- State variable h

$$h[n] = h[n-1] + \theta_h[n] \quad (21)$$

- Prediction

$$\mathbf{x}[n] = \mathbf{A}[n]\mathbf{x}[n-1] + \mathbf{w}[n] + \mathbf{B}[n]\mathbf{u}[n] \quad (22)$$

- Control vector $\mathbf{u}[n] = [0, 0, 0, 0, \theta_h[n]]^T$
- Control-input matrix

$$\mathbf{B}[n] = \begin{bmatrix} \mathbf{0}_{4 \times 5} \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

PROPOSED MODEL(CONT.)

- Observation

$$v_s[n] = \frac{\rho v_0}{4RA} \Delta \ell[n] \quad (23)$$

$$= \theta_1 \Delta \ell[n] \quad (24)$$

$$v_d[n] = \left(1 + \frac{R_2}{R_1}\right) \frac{d_{33}}{c_m} f' \quad (25)$$

$$= \theta_2 [k_2[n](\Delta \ell[n-1] - \Delta \ell[n-2]) + k_3[n](\Delta \ell[n] - 2\Delta \ell[n-1] + \Delta \ell[n-2])] \quad (26)$$

$$= \theta_2 [(k_2[n] + k_3[n])\Delta \ell[n] + (-k_2[n] - 2k_3[n])\Delta \ell[n-1] + k_3[n]\Delta \ell[n-2]] \quad (27)$$

$$\theta_1 = \frac{\rho v_0}{4RA}, \theta_2 = \left(1 + \frac{R_2}{R_1}\right) \frac{d_{33}}{c_m} \quad (28)$$

PROPOSED MODEL (MATRIX)

- Assumed process noise

$$\mathbf{w} = \Delta f \begin{bmatrix} 1 \\ \frac{1}{k_1+k_2+k_3} \\ 0 \\ 0 \\ 0 \end{bmatrix}, \Delta f \sim \mathcal{N}(0, \sigma_f^2) \quad (29)$$

- Covariance matrix

$$\mathbf{Q} = \mathbb{E}(\mathbf{w}^T \mathbf{w})$$

$$= \begin{bmatrix} 1 & \tilde{k} & 0 & 0 & 0 \\ \tilde{k} & \tilde{k}^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \sigma_f^2, \tilde{k} = \frac{1}{k_1 + k_2 + k_3}$$

- Prediction matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ \frac{1}{k+k_2+k_3} & \frac{k_2+2k_3}{k+k_2+k_3} & \frac{k_3}{k+k_2+k_3} & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (30)$$

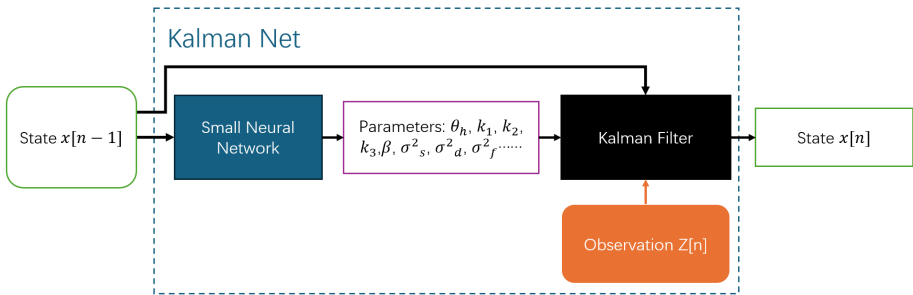
- Measurement to prediction matrix

$$\mathbf{H} = \begin{bmatrix} 0 & \theta_1 & 0 & 0 & 0 \\ 0 & \theta_2(k_2 + k_3) & -\theta_2 k_2 - 2\theta_2 k_3 & \theta_2 k_3 & 0 \end{bmatrix} \quad (31)$$

DEEP LEARNED KALMAN FILTER

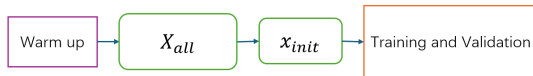
KALMAN NET

- The structure of deep learned Kalman filter.



ADDITIONAL TRAINING PROCESSES

- Warm up
 - Problem: Lack of temporal information
 - Solution: Input initial states
- Initialization
 - Parameters and weights
 - Nelder-Mead method



DEEP LEARNED KALMAN FILTER

OFFSET CORRECTION

- Motivation
 - The effect of state $h[n]$ on the final estimate is negligible
 - Outside the iteration
- Residual deformation $\Delta\ell_0$

$$f[n] = k_1(\Delta\ell[n] - \Delta\ell_0) + k_2(\Delta\ell[n-1] - \Delta\ell[n-2]) + k_3(\Delta\ell[n] - 2\Delta\ell[n-1] + \Delta\ell[n-2]) \quad (32)$$

- Estimated force in our proposed model

$$\hat{f}[n] = k_1\Delta\ell[n] + \dots \quad (33)$$

- Direct offset correction

$$\hat{f}'[n] = \hat{f}[n] - k_1\Delta\ell_0 \quad (34)$$

$$= \hat{f}[n] - k_1h[n-1] \quad (35)$$

DEEP LEARNED KALMAN FILTER

LOSS FUNCTION

- Primary loss
 - MAE (L1) loss

$$\mathcal{L}_{\ell_1}(f, \hat{f}) = \frac{1}{S} \sum_{i=1}^S |f[i] - \hat{f}[i]|. \quad (36)$$

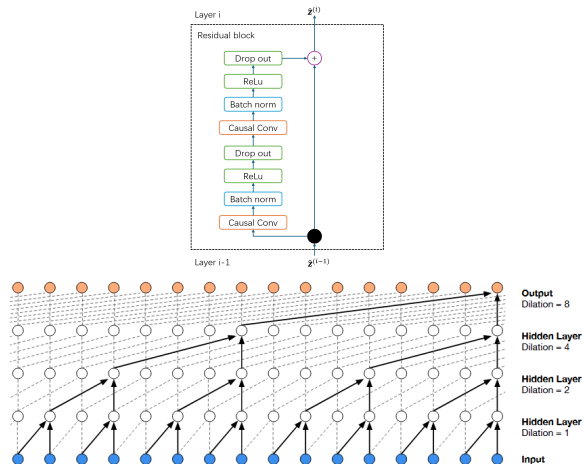
- Penalty: An additional loss
 - Better training direction
 - Faster convergence

$$\mathcal{P} = \lambda \frac{1}{S} \sum_i^S \mathbb{1}(|\ell_i| < \varepsilon) \text{ReLU}(\theta_h^{(i)}) \quad (37)$$

TCN

TEMPORAL CONVOLUTIONAL NEURAL NETWORK

- Characteristics
 - Causal: No information flows from future to past [S. Bai et al. '18]
 - Time invariant
- Architecture
 - Residual block: Prevent gradient vanishing [K. He et al. '16]
 - Dilated convolutional layers: Increase receptive field [V. Aäron et al. '16]



V. Aäron et al. '16

CONTENTS

1
2
3

Introduction

Data collection

Proposed Estimation Methods

4
5

Experimental Result

Conclusion

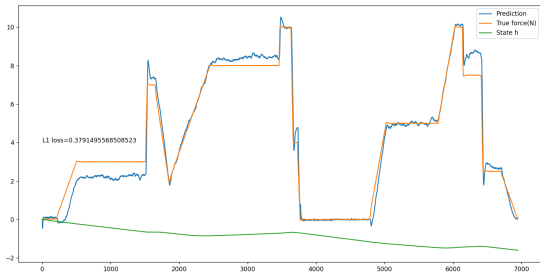
EXPERIMENTAL RESULT

Section 4

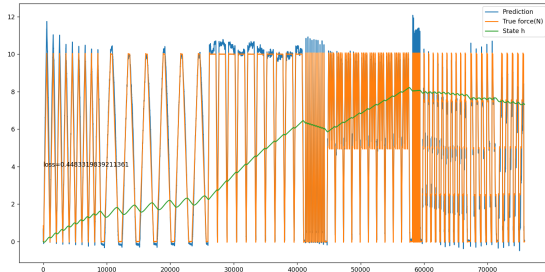
OVERVIEW

- Model list
 - DLKF-FFS
 - Base model
 - Offset correction
 - TCN
 - TCN-8
 - TCN-12
- Validation sequences
 - Long sequence with 73000 samples
 - Short sequence with 7000 samples

DLKF-FFS



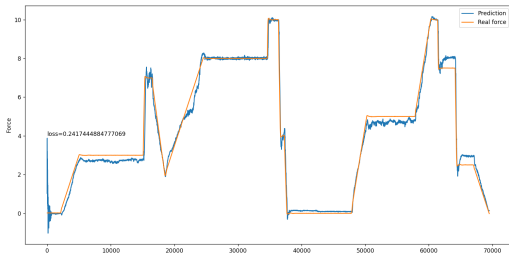
DLKF-FFS (Base model): Short validation



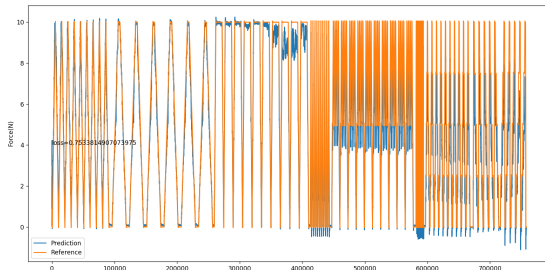
DLKF-FFS (Offset correction): Long validation

Method	Penalty	Training	Validation(Short)	Validation(Long)
Base model	Yes	0.36	0.38	0.59
	No	0.39	0.48	0.60
Base model (no v_d)	Yes	0.63	0.53	0.66
	No	0.57	0.46	0.60
Offset correction	Yes	0.45	0.39	0.45
	No	0.57	0.46	0.60

TCN



TCN-8: Short validation sequence



TCN-8: Long validation sequence

Model	Training	Validation (Short)	Validation (Long)
TCN-8	0.15	0.24	0.75
TCN-8 without v_d	0.22	0.30	1.11
TCN-12	0.14	0.34	0.74

COMPARISON

Aspect	TCN	DLKF-FFS
Short sequence performance	Better	Worse
Long sequence performance	Worse	Better
Hysteresis elimination	Worse	Better
Drift elimination	Better	Worse
Number of parameters	98.8k	1,099
Interpretability	Black box	Partially interpretable
Real-time implementation	Feasible	Feasible
Use of physical model knowledge	Model-agnostic	Required
Data requirements	Fully data-driven	Low requirements
Training process	Easier	Requires initialization and warm-up

CONTENTS

1

Introduction

2

Data collection

3

Proposed Estimation Methods

4

Experimental Result

5

Conclusion

CONCLUSION

Section 5

CONCLUSION

• Contribution

- Experimental design and data collection
- Implement two approaches
 - DLKF-FFS
 - TCN
- Exploring the feasibility of error reducing using piezoelectric signals

• Outlook

- Additional data sources
- More variables into the state model
 - Temperature
 - Piezoelectric coefficient
- Other variants of the Kalman filter
 - Extended Kalman filter

References

- [1] A. Matute, L. Paredes-Madrid, E. Gutierrez, and C. A. Parra Vargas, “Characterization of drift and hysteresis errors in force sensing resistors considering their piezocapacitive effect,” in 2017 IEEE SENSORS, Glasgow: IEEE, Oct. 2017, pp. 1–3.
- [2] O. B. Dali, “Ferroelectrets: From material science to energy harvesting and sensor applications,” en, Ph.D. dissertation, Technische Universität Darmstadt, Darmstadt, Jun. 2023, pp. xvi, 149.
- [3] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” arXiv, vol. arXiv/1803.01271, 2018.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Jun. 2016, pp. 770–778.
- [5] A. van den Oord et al., “WaveNet: A generative model for raw audio,” in Proc. 9th ISCA Workshop on Speech Synthesis Workshop, 2016, pp. 125–125.

Thank you for your attention!

Questions are welcome.

APPENDIX

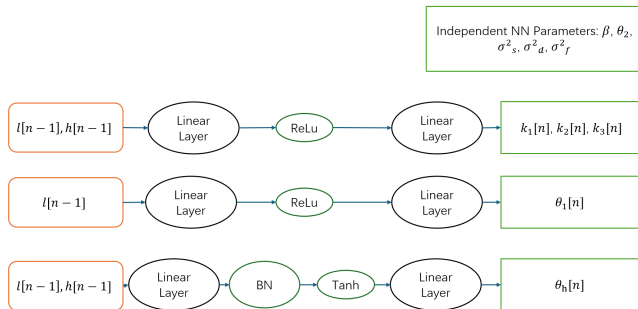
Section 6

SYNCHRONIZATION

- First-order derivative matching

$$\hat{G} = \arg \max_{0 \leq s_1 \leq \tau} \frac{dv_d}{dt} - \arg \max_{0 \leq s_2 \leq \tau} \frac{df}{dt}$$

KALMAN NET



PIEZOELECTRIC EFFECT AND THE ELECTRICAL RESISTANCE STRAIN GAUGE

$$v_o = \frac{f_d d_{33}}{c_m} \quad (38)$$

$$v_d = \left(1 + \frac{R_2}{R_1}\right) v_o = \left(1 + \frac{R_2}{R_1}\right) \frac{d_{33}}{c_m} f_d \quad (39)$$

$$\Delta R = \frac{\rho \Delta l}{A} \quad (40)$$

$$v_s = v_o \frac{\Delta R}{4R_0} \quad (41)$$

$$v_s = \frac{\rho v_o}{4R_0 A} \Delta l \quad (42)$$

LOSS FUNCTION

- MSE(L2) Loss

$$\mathcal{L}_{\ell_2}(f, \hat{f}) = \frac{1}{N} \sum_{i=1}^N \left(f[i] - \hat{f}[i] \right)^2 \quad (43)$$

- MAE(L1) Loss

$$\mathcal{L}_{\ell_1}(f, \hat{f}) = \frac{1}{N} \sum_{i=1}^N \left| f[i] - \hat{f}[i] \right| \quad (44)$$

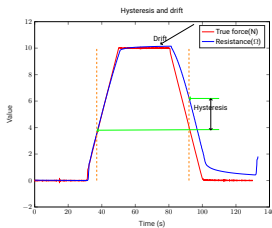
- Stochastic Gradient Descent (SGD)

$$\mathbf{w}^*, \mathbf{b}^* = \arg \min_{\mathbf{w}, \mathbf{b}} \mathcal{L}(f, \hat{f}) \quad (45)$$

REFERENCE DRIFT AND HYSTERESIS ERROR

$$\text{DER} = 100\% \frac{(r_f - r_i)}{r_i}. \quad (46)$$

$$\text{HER} = 100\% \frac{(r_u - r_l)}{r_m}. \quad (47)$$



	HER	DER(per second)
Maximum	17.36%	0.99‰
Average	14.26%	0.88‰

EXPERIMENTAL RESULT

HYSTERESIS AND DRIFT

- TCN

	HER	DER
Maximum	12.38%	0.73‰
Average	8.04%	0.62‰
Improvement compared to the reference	43.58%	29.22%

- DLKF-FFS

	HER	DER(per second)
Maximum	10.71%	1.11‰
Average	7.90%	0.72‰
Improvement compared to the reference	44.62%	17.9%

REAL TIME IMPLEMENTATION

For the TCN model, the implementation steps are as follows:

- A low-pass filter is applied to pre-process the input data.
- Warm up: At the beginning, in order to ensure good results, it is necessary to wait for enough input samples, e.g. 40s or 20k samples.
- Every time after a refresh period τ , read the last 20k sample points to do the prediction by a pre-trained model(it takes about 0.05s), find the start and end time point of the new input, and make the corresponding output.

Meanwhile, the application process for DLKF-FFS is as follows:

- Data preprocessing of the input signal as above.
- No warm-up is required because DLKF-FFS always has consistent output for inputs of arbitrary length.
- Every time a refresh cycle τ passes, the input signal and state $\mathbf{x}_{n-1}$ are provided to the pre-trained model, and the result is predicted which takes about 0.038τ seconds. Output the results and record the ending state $\mathbf{x}_n$.